

Table of Contents

1 LAB COMPLET.....	2
2 Objectif.....	2
2.1 Objectif du lab.....	2
2.2 Résultats attendus.....	2
2.3 Architecture cible.....	3
3 Prérequis.....	3
3.1 ansible et Terraform.....	3
3.2 AWS configuré :.....	4
3.3 Clé SSH :.....	4
3.3.1 Méthode 1 (RECOMMANDÉE) — via AWS.....	4
3.3.2 Autre Option : clé dans le projet (lab uniquement).....	5
Structure :	5
3.3.3 Méthode 2 (optionnelle) — générer soi-même la clef.....	6
4 Structure du projet.....	7
5 Terraform.....	8
6 Lancer Terraform.....	13
7 Ansible.....	16
7.1 inventory.ini.....	16
7.2 ansible.cfg.....	16
7.3 Playbook.....	17
7.4 Rôle nginx.....	17
7.5 fichier index.....	18
7.6 handler.....	18
8 Exécuter Ansible.....	18
8.1 tester inventory.....	18
8.2 Déployer nginx.....	19
9 test load balancer.....	20
9.1. Test http.....	20
9.2 Tests CLI AWS.....	21
9.3 👉 dans la console AWS :.....	23

10 Optionnel : Générer le schéma de la topologie.....	24
11 terraform destroy.....	27

1 LAB COMPLET

WSL + Terraform + AWS + Ansible + rôle nginx

2 Objectif

2.1 Objectif du lab

Ce lab a pour objectif de mettre en place une chaîne complète d'automatisation **Infrastructure + Configuration** en utilisant **Terraform, AWS et Ansible**.

L'objectif est de comprendre comment :

- Provisionner une infrastructure cloud sur **AWS** avec Terraform
- Configurer automatiquement les serveurs avec Ansible
- Appliquer les bonnes pratiques Ansible (roles, variables, templates, handlers)
- Créer un déploiement reproductible, modulaire et maintenable

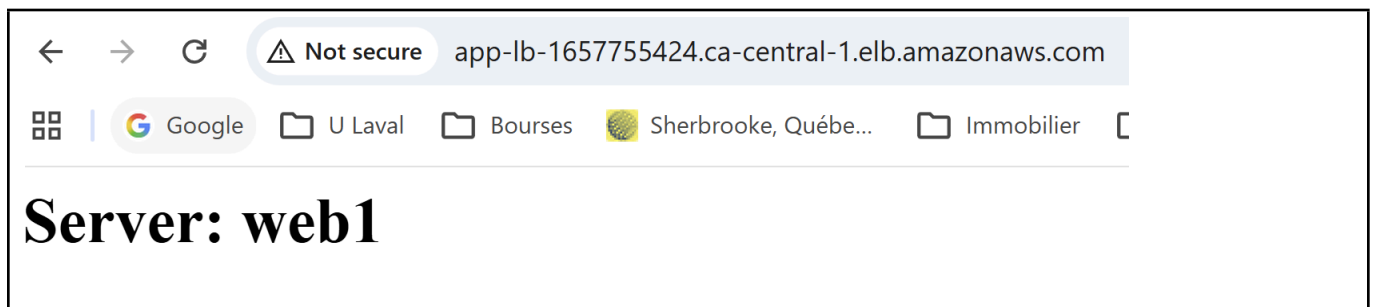
2.2 Résultats attendus

À la fin du lab, 2 instances EC2 de AWS feront offices de serveur web et seront:

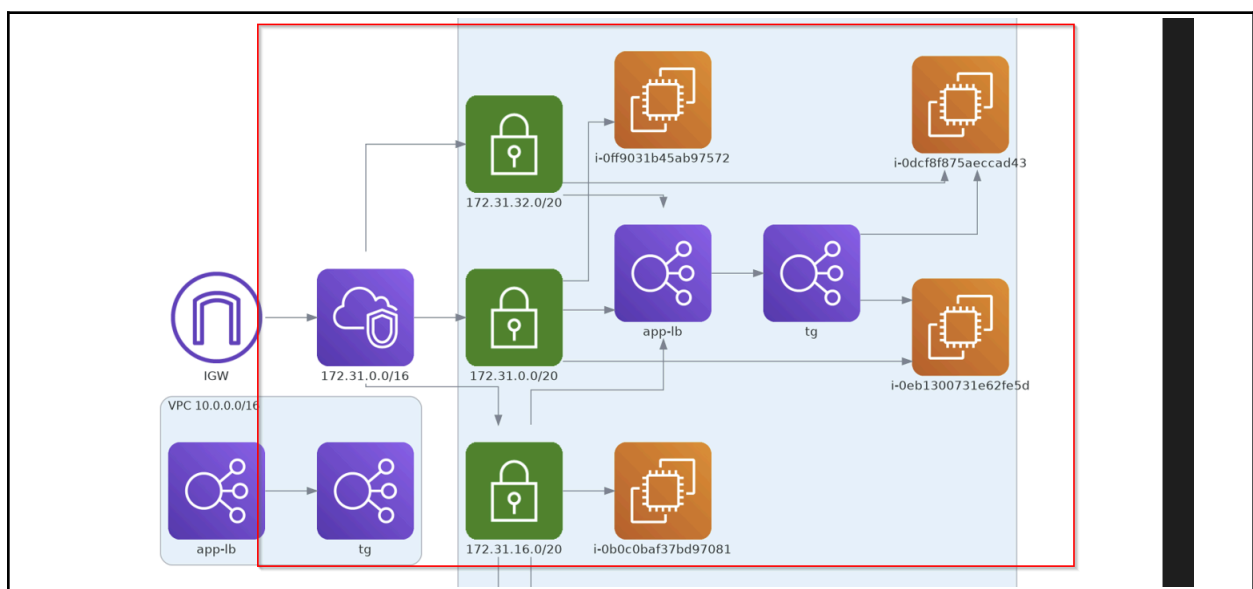
- créées automatiquement avec Terraform
- accessibles en SSH via une clé privée
- configurée avec Ansible
- exposées via un serveur web (Nginx)
- Les deux instances EC2 seront rattachées à un load balancer

Une page web sera accessible et affichera alternativement la page du serveur web 1 ou celle du serveur web 2, ici par exemple, c'est celle du serveur web 1.

Le load balancer répartit sa charge sur les deux serveurs web (les deux instances EC2 de AWS).



2.3 Architecture cible



3 Prérequis

3.1 ansible et Terraform

Dans WSL (/home/henri) :

Si terraform pas installé :
Installer le package terraform dans Ubuntu

```
henri@Henri:~$  
henri@Henri:~$ sudo apt-get update  
sudo apt-get install terraform
```

Ensuite , vérifier la version de terraform
terraform -v

```
henri@Henri:~$ terraform -v  
Terraform v1.14.8  
on linux_amd64  
henri@Henri:~$  
henri@Henri:~$
```

Ensuite , si ansible non installé, alors l'installer dans Ubuntu

Ensuite : vérifier la version de ansible
ansible --version

```
henri@Henri:~$  
henri@Henri:~$ ansible --version  
ansible [core 2.16.3]  
  config file = None  
  configured module search path = ['/home/henri/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']  
  ansible python module location = /usr/lib/python3/dist-packages/ansible  
  ansible collection location = /home/henri/.ansible/collections:/usr/share/ansible/collections  
  executable location = /usr/bin/ansible  
  python version = 3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)  
  jinja version = 3.1.2  
  libyaml = True  
henri@Henri:~$
```

3.2 AWS configuré :

aws configure

3.3 Clé SSH :

3.3.1 Méthode 1 (RECOMMANDÉE) — via AWS

👉 C'est la méthode la plus simple pour débiter

1) Créer une key pair dans AWS

Dans AWS :

Va dans EC2

Menu gauche → Key Pairs

Clique Create key pair

Paramètres :

Name: mykeypair

Type: RSA

Format: .pem

👉 Cliquez Create

2] Téléchargement

👉 AWS télécharge automatiquement :

mykeypair.pem

⚠️ IMPORTANT :

tu ne peux le télécharger qu'une seule fois

3] Déplacer la clé

Dans ton terminal :

```
mkdir -p ~/.ssh
```

```
mv ~/Downloads/ mykeypair.pem ~/.ssh/
```

4] Sécuriser la clé (obligatoire)

```
chmod 400 ~/.ssh/ mykeypair.pem
```

👉 Sinon SSH refusera la connexion

Tester

```
ssh -i ~/.ssh/ mykeypair.pem ec2-user@IP_PUBLIC
```

🔗 Lien avec Terraform

Dans ton main.tf :

```
key_name = " mykeypair "
```

👉 ⚠️ le nom doit être EXACTEMENT le même que dans AWS

🔗 Lien avec Ansible

Inventory :

```
[web]
```

```
IP_PUBLIC ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/ mykeypair.pem
```

3.3.2 Autre Option : clé dans le projet (lab uniquement)

Structure :

```
project/
├── terraform/
├── ansible/
├── keys/
│   └── mykeypair
```

Permissions

```
chmod 400 keys/mykeypair
```



Ansible

```
ansible_ssh_private_key_file=../keys/mykeypair
```



SSH

```
ssh -i ../keys/mykeypair ec2-user@IP
```

3.3.3 Méthode 2 (optionnelle) — générer soi-même la clef



plus avancé (utile plus tard)

```
ssh-keygen -t rsa -b 2048 -f ~/.ssh/my-keypair
```



ça crée :

```
~/.ssh/my-keypair
```

```
~/.ssh/my-keypair.pub
```

Mais dans AWS il faut importer la clé → plus complexe



Vérifier

```
ls ~/.ssh/my-keypair*
```



Permissions

```
chmod 400 ~/.ssh/my-keypair
```



2. Importer dans AWS

Étapes :

Va dans EC2

Menu gauche → Key Pairs

Clique Import key pair

Paramètres :

Name: my-keypair ⚠ (important pour Terraform)

Public key content :



copie le contenu de :

```
cat ~/.ssh/my-keypair.pub
```



ça ressemble à :

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQ... user@host
```



colle ça dans AWS



clique Import key pair

3. Utilisation avec Terraform

Dans ton main.tf :

```
key_name = "my-keypair"
```

👉 doit matcher EXACTEMENT

4. Utilisation avec Ansible

[web]

```
IP_PUBLIC ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/my-keypair
```

4 Structure du projet

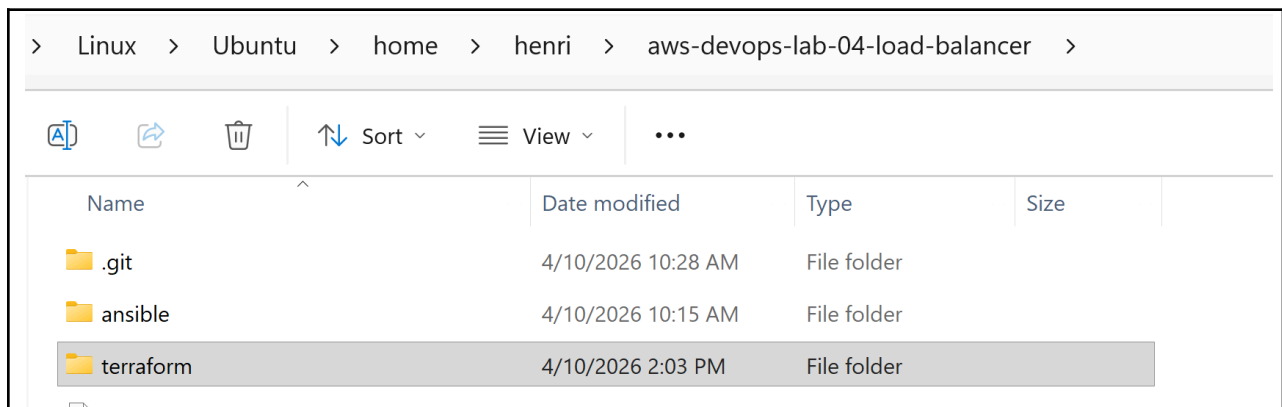
Se positionner sur le répertoire utilisateur racine

Ici c'est /home/henri/

```
cd ~
```

```
mkdir -p aws-devops-lab-04-load-balancer/{terraform,ansible/playbooks,ansible/roles/nginx/tasks}
```

```
cd aws-devops-lab-04-load-balancer
```



```
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── ec2.tf
│   ├── alb.tf
│   └── security_groups.tf
├── ansible/
│   ├── inventory.ini
│   ├── playbook.yml
│   └── roles/
│       └── nginx/
└── README.md
```

5 Terraform

👉 terraform/main.tf

```
provider "aws" {
  region = "ca-central-1"
}

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}
```

👉 Créer deux EC2 : ec2.tf

```
resource "aws_instance" "web" {
  count      = 2
  ami       = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type
```



```
subnet_id = data.aws_subnets.default.ids[count.index]

associate_public_ip_address = true # 👉 IMPORTANT

vpc_security_group_ids = [aws_security_group.web_sg.id]

key_name = var.key_name

tags = {
  Name = "web-${count.index + 1}"
}
```

On mets `associate_public_ip_address = true` car on veut une IP publique pour les EC2

```

. . . . .
. . . . .
))))))))))

```

On utilise count.index dans l'expression ci-dessous afin d'avoir 2 subnets . le load balancer doit avoir une instance EC2 dans un subnet et l'Autre EC2 dans un subnet differend.

On aura : `data.aws_subnets.default.ids[0]` et `data.aws_subnets.default.ids[1]`

```
subnet id = data.aws_subnets.default.ids[count.index]
```

web1 = healthy ✓

web2 = healthy ✓

Sinon si les deux instances EC2 sont dans le meme subnet, ce qui se passera est qu'une seule instance sera fonctionnelle (dans l'état healthy) et la seconde non fonctionnelle (unhealthy)

A ce moment la il faudrait utiliser l'Expression suivante : sans count.index

```
subnet id = data.aws_subnets.default.ids
```

web1 = healthy ✓

web2 = unhealthy ❌

👉 Security group (Ec2 + LB) : security_groups.tf

```
# SG pour ALB
resource "aws_security_group" "alb_sg" {
  name = "alb_sg"

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```

    egress {
      from_port = 0
      to_port   = 0
      protocol  = "-1"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }

# SG pour EC2
resource "aws_security_group" "web_sg" {
  name = "web_sg"

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    security_groups = [aws_security_group.alb_sg.id]
  }

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["YOUR_IP/32"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

```

Dans le fichier ci-dessus : security_groups.tf remplacer YOUR_IP/32 dans expression : cidr_blocks = ["YOUR_IP/32"] par l'IP de votre ordinateur afin que vous puissiez vous connecter en ssh sur les EC2. Par exemple faire : curl ifconfig.me
Puis remplacer YOUR_IP/32 par votre adresse IP

Sinon , comme c'est un lab , utiliser simplement : cidr_blocks = ["0.0.0.0/0"]

👉 Load balancer : alb.tf

```

resource "aws_lb" "app_lb" {
  name           = "app-lb"
  load_balancer_type = "application"
  subnets       = [aws_subnet.public.id]

  security_groups = [aws_security_group.alb_sg.id]
}

resource "aws_lb_target_group" "tg" {
  name     = "tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.main.id

  health_check {
    path = "/"
    protocol = "HTTP"
    matcher = "200"
    interval = 30
    timeout = 5
    healthy_threshold = 2
    unhealthy_threshold = 2
  }
}

resource "aws_lb_target_group_attachment" "tg_attach" {
  count      = 2
  target_group_arn = aws_lb_target_group.tg.arn
  target_id   = aws_instance.web[count.index].id
  port       = 80
}

resource "aws_lb_listener" "listener" {
  load_balancer_arn = aws_lb.app_lb.arn
  port             = 80
  protocol         = "HTTP"

  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.tg.arn
  }
}

```

👉 Output terraform : outputs.tf

```

output "instance_ips" {
  value = aws_instance.web[*].public_ip
}

```

```
}

output "alb_dns" {
  value = aws_lb.app_lb.dns_name
}
```

👉 terraform/variables.tf

```
variable "key_name" {
  description = "AWS key pair name - Nom de la key pair AWS "
  type = string
}

variable "instance_type" {
  description = "Type EC2"
  type       = string
  default    = "t3.micro"
}
```

👉 Créer terraform.tfvars

```
key_name = "mykeypair"
instance_type = "t3.micro"
```

👉 Créer outputs.tf

```
output "instance_ips" {
  description = "Public IPs des EC2"
  value      = aws_instance.web[*].public_ip
}

output "alb_dns" {
  description = "DNS du Load Balancer"
  value      = aws_lb.app_lb.dns_name
}
```

6 Lancer Terraform

👉 cd aws-devops-lab-04-load-balancer

terraform plan

terraform init

terraform apply

Terraform init :

```
henri@Henri:~/Lab-#3-devops-lab/terraform$
henri@Henri:~/Lab-#3-devops-lab/terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.40.0...
- Installed hashicorp/aws v6.40.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
henri@Henri:~/Lab-#3-devops-lab/terraform$
```

👉 Terraform plan

```
+ name_prefix      = (known after apply)
+ owner_id         = (known after apply)
+ region           = "ca-central-1"
+ revoke_rules_on_delete = false
+ tags_all         = (known after apply)
+ vpc_id           = (known after apply)
}

Plan: 9 to add, 0 to change, 0 to destroy.
```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

```
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
```

👉 Terraform apply

On crée 4 ressources.

```
Enter a value: yes
aws_instance.web[1]: Creating...
aws_instance.web[0]: Creating...
aws_instance.web[0]: Still creating... [00m11s elapsed]
aws_instance.web[1]: Still creating... [00m11s elapsed]
aws_instance.web[1]: Creation complete after 14s [id=i-0bcde3d6b50d6d87a]
aws_instance.web[0]: Creation complete after 14s [id=i-0a9f1740ed7857e04]
aws_lb_target_group_attachment.tg_attach[0]: Creating...
aws_lb_target_group_attachment.tg_attach[1]: Creating...
aws_lb_target_group_attachment.tg_attach[1]: Creation complete after 0s [id=arn:aws:elasticloadbalancing:ca-central-1:760606734887:targetgroup/tg/591d126a86d12d4d,i-0bcde3d6b50d6d87a,80]
aws_lb_target_group_attachment.tg_attach[0]: Creation complete after 0s [id=arn:aws:elasticloadbalancing:ca-central-1:760606734887:targetgroup/tg/591d126a86d12d4d,i-0a9f1740ed7857e04,80]
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
```

👉 Terraform output

Cette commande permet de lister toutes les variables qui sont définies en output

```
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$ terraform output
alb_dns = "app-lb-1657755424.ca-central-1.elb.amazonaws.com"
instance_ips = [
    "3.99.81.169",
    "15.223.223.185",
]
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
```

👉 Terraform state list

On veut lister toutes les ressources AWS qui ont été créées

```
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$ terraform state list
data.aws_ami.amazon_linux
data.aws_subnets.default
data.aws_vpc.default
aws_instance.web[0]
aws_instance.web[1]
aws_lb.app_lb
aws_lb_listener.listener
aws_lb_target_group.tg
aws_lb_target_group_attachment.tg_attach[0]
aws_lb_target_group_attachment.tg_attach[1]
aws_security_group.alb_sg
aws_security_group.web_sg
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
```

👉 Test SSH :

Test ssh afin de vérifier si on peut joindre les instances EC2 qui ont été créées dans l'étape précédente par terraform

Récupérer les deux adresses IP des deux instances EC2.

Puis :

```
ssh -i ~/.ssh/mykeypair.pem ec2-user@IP1
```

```
et ssh -i ~/.ssh/mykeypair.pem ec2-user@IP2
```

```
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$ ssh -i ~/.ssh/mykeypair.pem ec2-user@3.99.81.169
The authenticity of host '3.99.81.169 (3.99.81.169)' can't be established.
ED25519 key fingerprint is SHA256:MTUJe86lyf3G0I8yIpfm055PIhvV2vToRWhZmGUJ7IY.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? ye
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '3.99.81.169' (ED25519) to the list of known hosts.

#_
~\#####_      Amazon Linux 2
nn\#####\
nn\###|
nn\#/
nnV~'--->
nnn
nn.-.
_/_/_/_/_/_/_/_
_/m/'

AL2 End of Life is 2026-06-30.

A newer version of Amazon Linux is available!

Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-172-31-40-152 ~]$ exit
logout
Connection to 3.99.81.169 closed.
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
```

En cas de problèmes sur les clefs refaire une régénération de clefs ;avec les commandes :

```
rm ~/.ssh/known_hosts
```

```
ssh-keygen -R IP1
```

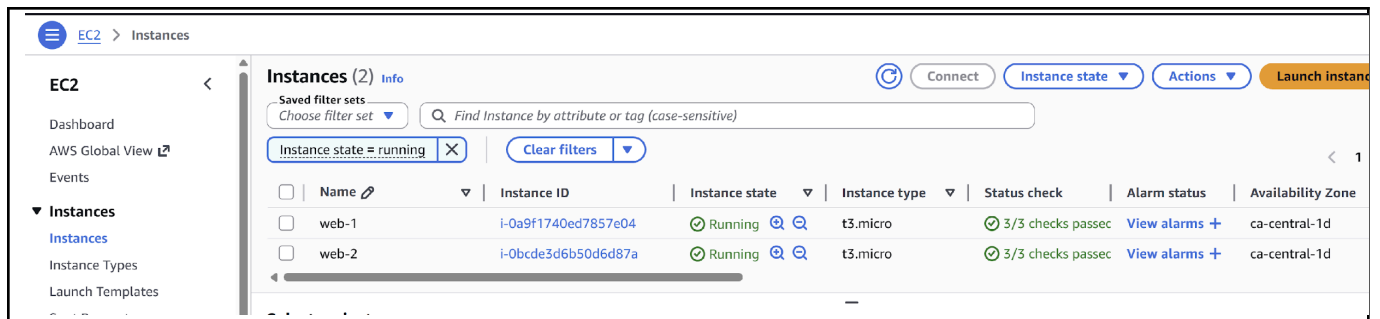
```
ssh-keygen -R IP2
```

exemple : ci-dessous

```
henri@Henri:~/aws-devops-lab-04-load-balancer/ansible$ ssh-keygen -R 3.99.81.169
# Host 3.99.81.169 found: line 13
# Host 3.99.81.169 found: line 14
/home/henri/.ssh/known_hosts updated.
Original contents retained as /home/henri/.ssh/known_hosts.old
henri@Henri:~/aws-devops-lab-04-load-balancer/ansible$ ssh-keygen -R 15.223.223.185
Host 15.223.223.185 not found in /home/henri/.ssh/known_hosts
henri@Henri:~/aws-devops-lab-04-load-balancer/ansible$
henri@Henri:~/aws-devops-lab-04-load-balancer/ansible$
```

👉 Test dans AWS console

Dans AWS, on vérifie la présence des instances EC2 qui ont été créées



7 Ansible

cd aws-devops-lab-04-load-balancer/ansible

7.1 inventory.ini

👉 Créer un fichier, intitulé : inventory.ini

remplacer IP par les deux adresses IP1 Et IP2

```
[web]
web1 ansible_host=IP1 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/mykeypair.pem
web2 ansible_host=IP2 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/mykeypair.pem
```

7.2 ansible.cfg

Créer un fichier ansible.cfg

```
[defaults]
host_key_checking = False
inventory = inventory.ini
```

Définit l'inventary par défaut : inventory = inventory.ini

👉 donc on peut lancer la commande : ansible-playbook playbook.yml
ce sans préciser -i inventory.ini

Si on rajoute le username et le chemin de clef dans le fichier de configuration : ansible.cfg alors il n'est pas nécessaire de les rajouter dans le fichier inventory.ini

Exemple :

Ansible.cfg :

```
[defaults]
host_key_checking = False
inventory = inventory.ini
remote_user = ec2-user
private_key_file = ~/.ssh/mykeypair.pem
```

Inventory.ini :

```
[web]
web1 ansible_host=IP1
web2 ansible_host=IP2
```

7.3 Playbook

👉 playbooks/site.yml

```
- hosts: web
  become: yes
  roles:
    - nginx
```

7.4 Rôle nginx

👉 roles/nginx/tasks/main.yml

```
- name: Enable nginx repo
  shell: amazon-linux-extras enable nginx1
  changed_when: false

- name: Install nginx
  yum:
    name: nginx
    state: present

- name: Deploy web page
  template:
    src: index.html.j2
    dest: /usr/share/nginx/html/index.html
  notify: restart nginx
```

```
- name: Ensure nginx is running
  service:
    name: nginx
    state: started
    enabled: true
```

7.5 fichier index

Créer le répertoire templates et le fichier index

👉 roles/nginx/templates/index.html.j2

Contenu du fichier index.html.j2 :

```
<h1>Server: {{ inventory_hostname }}</h1>
```

7.6 handler

👉 roles/nginx/handlers/main.yml

Contenu du fichier main.yml qui est dans le répertoire handler :

```
- name: restart nginx
  service:
    name: nginx
    state: restarted
```

8 Exécuter Ansible

8.1 tester inventory

```
cd aws-devops-lab-04-load-balancer/ansible
ansible all -m ping
```

```
henri@Henri:~/aws-devops-lab-04-load-balancer/ansible$ ansible all -m ping
[WARNING]: Platform linux on host web2 is using the discovered Python interpreter at /usr/bin/python3.7, but future
installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-
core/2.16/reference_appendices/interpreter_discovery.html for more information.
web2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.7"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Platform linux on host web1 is using the discovered Python interpreter at /usr/bin/python3.7, but future
installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-
core/2.16/reference_appendices/interpreter_discovery.html for more information.
web1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.7"
  },
  "changed": false,
  "ping": "pong"
}
henri@Henri:~/aws-devops-lab-04-load-balancer/ansible$
```

Le resultat doit etre ‘ping’: ‘pong’

Et on doit avoir :

web1 | SUCCESS

web2 | SUCCESS

Donc ici, c’est correct.

8.2 Déployer nginx

cd aws-devops-lab-04-load-balancer/ansible

ansible-playbook playbooks/site.yml -i inventory.ini

ou bien :

ansible-playbook playbooks/site.yml

```

henri@Henri:~/aws-devops-lab-04-load-balancer/ansible$ ansible-playbook playbooks/site.yml

PLAY [web] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host web2 is using the discovered Python interpreter at /usr/bin/python3.7, but future
installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-
core/2.16/reference_appendices/interpreter_discovery.html for more information.
ok: [web2]
[WARNING]: Platform linux on host web1 is using the discovered Python interpreter at /usr/bin/python3.7, but future
installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-
core/2.16/reference_appendices/interpreter_discovery.html for more information.
ok: [web1]

TASK [nginx : Enable nginx repo] *****
ok: [web2]
ok: [web1]

TASK [nginx : Install nginx] *****
changed: [web2]
changed: [web1]

TASK [nginx : Deploy web page] *****
changed: [web2]
changed: [web1]

TASK [nginx : Ensure nginx is running] *****
changed: [web1]
changed: [web2]

RUNNING HANDLER [nginx : restart nginx] *****
changed: [web1]
changed: [web2]

PLAY RECAP *****
web1      : ok=6    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
web2      : ok=6    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

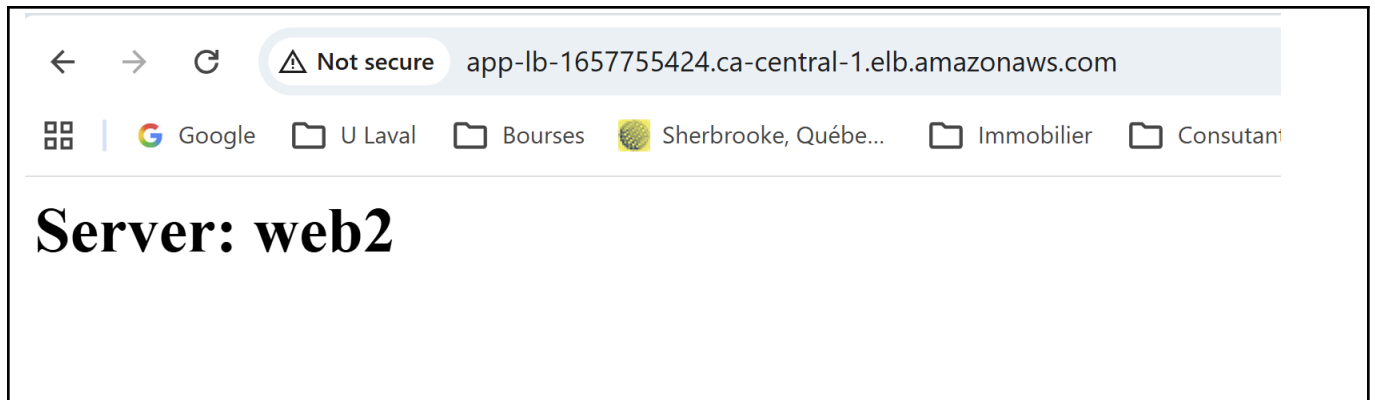
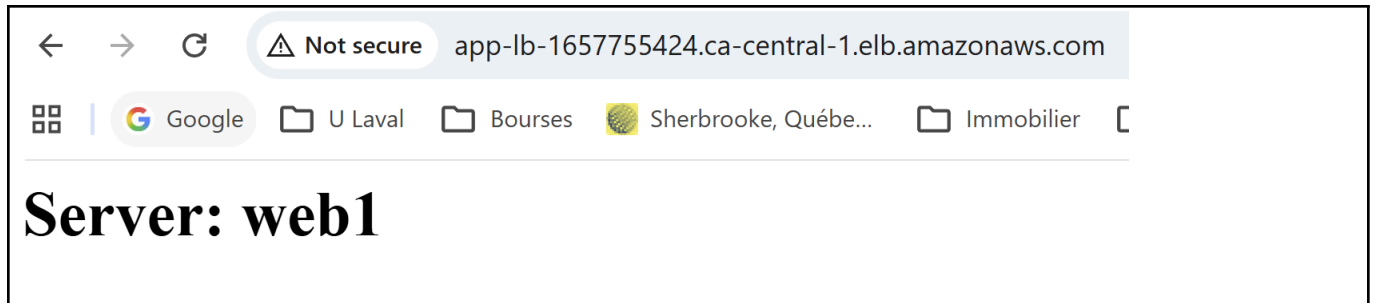
9 test load balancer

9.1. Test http

http://ALB_DNS

soit ici :

<http://app-lb-1657755424.ca-central-1.elb.amazonaws.com>



On constate que une fois on a web1 la fois d'Après après un refresh on a web2

Ainsi de suite.

Donc le load balancer balance alternativement le trafic entre les deux instances ip

9.2 Tests CLI AWS

👉 Test CLI :

Lister les ALB :

```
aws elbv2 describe-load-balancers
```

👉 filtrer l'ALB :

```
aws elbv2 describe-load-balancers --query "LoadBalancers[*].DNSName"
```

```
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$  
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$ aws elbv2 describe-load-balancers --query "LoadBalancers[*].DNSName"  
[  
  "app-lb-1657755424.ca-central-1.elb.amazonaws.com"  
]  
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
```

👉 Lister les targets groups :

aws elbv2 describe-target-groups

👉 récupérer l'ARN du TG :

aws elbv2 describe-target-groups --query "TargetGroups[*].TargetGroupArn"

l'ARN récupéré est ici : arn:aws:elasticloadbalancing:ca-central-1:ID:targetgroup/tg/591d126a86d12d4d

ID représente le numéro d'ID du compte AWS

```
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$  
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$ aws elbv2 describe-target-groups --query "TargetGroups[*].TargetGroupArn"  
[  
  "arn:aws:elasticloadbalancing:ca-central-1:██████████:targetgroup/tg/591d126a86d12d4d"  
]  
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$  
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
```

👉 vérifier la santé des instances:

aws elbv2 describe-target-health --target-group-arn <TG_ARN>

<TG_ARN> représente l'ARN du target de l'ALB. Qui a été obtenu précédemment.

```
henri@Henri: ~/aws-devops-l × + v
{
  "TargetHealthDescriptions": [
    {
      "Target": {
        "Id": "i-0a9f1740ed7857e04",
        "Port": 80
      },
      "HealthCheckPort": "80",
      "TargetHealth": {
        "State": "healthy"
      },
      "AdministrativeOverride": {
        "State": "no_override",
        "Reason": "AdministrativeOverride.NoOverride",
        "Description": "No override is currently active on target"
      }
    },
    {
      "Target": {
        "Id": "i-05c16e52780f5d19c",
        "Port": 80
      },
      "HealthCheckPort": "80",
      "TargetHealth": {
        "State": "healthy"
      },
      "AdministrativeOverride": {
        "State": "no_override",
        "Reason": "AdministrativeOverride.NoOverride",

```

On voit que les deux instances target sont healthy

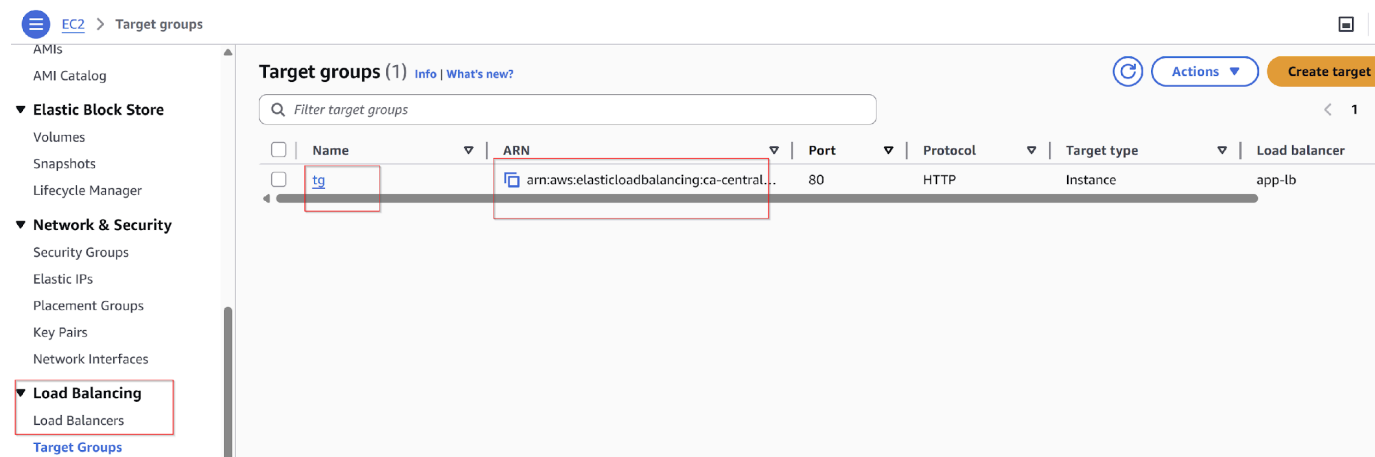
👉 Vérifier les instances :

```
aws ec2 describe-instances --query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress,State.Name]"
```

👉 voir les règles :

```
aws ec2 describe-security-groups --query "SecurityGroups[*].IpPermissions"
```

9.3 👉 dans la console AWS :



```
cd ~/aws-topology-1
source venv/bin/activate
python topology.py
```



```

henri@Henri:~/aws-topology-v1$ pwd
/home/henri/aws-topology-v1
henri@Henri:~/aws-topology-v1$ source venv/bin/activate
(venv) henri@Henri:~/aws-topology-v1$ python topology.py
(venv) henri@Henri:~/aws-topology-v1$ ls -als
total 340
 4 drwxr-xr-x  4 henri henri   4096 Apr 11 13:54 .
 4 drwxr-x--- 19 henri henri   4096 Apr 11 12:40 ..
 4 drwxr-xr-x  8 henri henri   4096 Apr 11 12:41 .git
212 -rw-r--r--  1 henri henri 216848 Apr 11 13:54 aws_full_topology.png
 72 -rw-r--r--  1 henri henri  70367 Apr 10 21:32 aws_vpc_network_topology.png
  4 -rw-r--r--  1 henri henri    25 Apr 11 11:17 aws_vpc_network_topology.png:Zone.Identifier
  4 -rw-r--r--  1 henri henri    53 Feb 10 23:42 requirements.txt
  4 -rw-r--r--  1 henri henri    25 Apr 11 11:17 requirements.txt:Zone.Identifier
  8 -rw-r--r--  1 henri henri   7148 Apr 11 11:56 topology.py
  4 -rw-r--r--  1 henri henri    25 Apr 11 11:17 topology.py:Zone.Identifier
20 drwxr-xr-x  5 henri henri  20480 Apr 11 11:28 venv
(venv) henri@Henri:~/aws-topology-v1$

```

Fichier résultat : aws_full_topology.png



Le ALB comme on peut le voir dans la figure ci-dessous (obtenu via la console AWS) est en fait mappé sur 3 subnets , sur 3 zones : ca-central-1a, ca-central-1b, ca-central-1d . Qui correspondent aux subnets : 172.31.0.0/20 . 172.31.16.0.20. 172.31.32.0.20

Ceci est confirmé, également par le graphique, obtenu par l’outil python , que nous avons installé précédemment.

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

▼ Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

▼ Load Balancing

Load Balancers

Target Groups

Trust Stores

Network mapping [Info](#)

Targets in the listed zones and subnets are available for traffic from the load balancer using the IP addresses shown.

VPC

[vpc-0518bdeefb96a886d](#)

IPv4 VPC CIDR: 172.31.0.0/16

IPv6 VPC CIDR: -

Load balancer IP address type

IPv4

IP pools

-

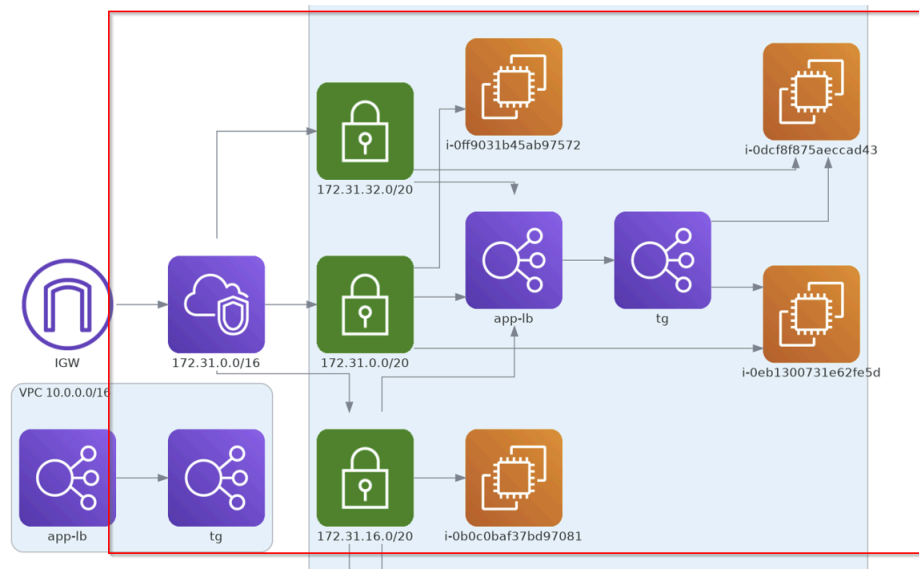
Availability Zones and subnets [Info](#)

Including two or more Availability Zones, and corresponding subnets, increases the fault tolerance of your applications.

Zone	Subnet	Private IPv4 address
ca-central-1d (cac1-az4)	subnet-04ab7f22ddced88f9	Assigned from CIDR 172.31.32.0/20
ca-central-1b (cac1-az2)	subnet-07b548eec9ff23574	Assigned from CIDR 172.31.0.0/20
ca-central-1a (cac1-az1)	subnet-06a2296f8b227b258	Assigned from CIDR 172.31.16.0/20

Avec un grossissement de l'image, on voit l'icone des ALB, et celle du security group ainsi que les instances EC2 qui sont représentées. On note la présence des 2 instances EC2 qui sont pluggées sur le load balancer : app-lb via le target group : tg

A la fois le graphique montre que les 2 instances EC2 appartiennent à leurs subnets respectifs : 172.31.32.0/20 et 172.31.16.0/20 et montrent aussi leur attachement au load balancer : app-lb via le target group : tg



A la fin du lab, si on en a plus besoin. On peut détruire toutes les ressources qui avaient été précédemment créées dans ce lab.

```
cd ~/Lab-#3-devops-lab/terraform  
terraform destroy
```

```
aws_security_group.web_sg: Destruction complete after 1s  
aws_security_group.alb_sg: Destroying... [id=sg-00a7af6d2fc879bdc]  
aws_security_group.alb_sg: Destruction complete after 1s
```

```
Destroy complete! Resources: 9 destroyed.
```

```
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$  
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$  
henri@Henri:~/aws-devops-lab-04-load-balancer/terraform$
```